REST API Data Explorer

Today's Learning Report — REST APIs, GET Requests & JSON

1. Project Overview

We built a Python-based REST API Data Explorer. The application communicates with an external REST API using the requests library, retrieves user information through GET requests, receives JSON data, extracts selected fields, processes the information, and saves the result locally.

2. Project Structure

REST_API_Data_Explorer/
■■■ api_client.py — reusable REST API client
■■■ main.py — application logic and data processing
■■■ requirements.txt — project dependency
■■■ data/users.json — processed API data

3. What We Learned

Concept	What We Learned
REST API	How applications communicate with external services through HTTP-based APIs.
GET Request	How to retrieve resources using requests.get().
HTTP Status	How response status codes indicate request success or failure.
JSON	How response.json() converts API data into Python objects.
Nested JSON	How to extract values such as address.city and company.name.
API Client	How to create a reusable APIClient class for API communication.
Error Handling	How to handle timeout, connection, HTTP, and JSON errors.
Data Processing	How to extract and transform only required information.
Persistence	How to save processed API data to a JSON file.
Dependencies	Why requirements.txt should contain packages required by the project.

4. API Workflow

Python Application → **GET Request** → **External REST API** → **HTTP Response** → **JSON Data** → **Data Extraction** → **Data Processing** → **Local JSON Storage**

5. Main Technical Concepts

requests.get() sends the GET request.
response.raise_for_status() detects unsuccessful HTTP responses.
response.json() converts JSON response data into Python objects.
timeout=10 prevents indefinite waiting.
APIClient provides reusable API communication logic.
json.dump() stores processed data in JSON format.

6. Project Features

• Connects to an external REST API.

• Retrieves user records through GET endpoints.

• Retrieves a specific user by ID.

• Extracts ID, name, username, email, city, and company.

• Handles common API failures.

• Processes and saves selected API information.

• Uses separation between API communication and application logic.

7. Expected Outcomes Achieved

Expected Outcome	Status
Understand how applications retrieve data from external services.	**Achieved**
Use requests to send a GET request.	**Achieved**
Receive JSON data from an API.	**Achieved**
Extract selected information from JSON.	**Achieved**
Understand REST API communication.	**Achieved**

8. Key Takeaway

The project demonstrated the complete API integration workflow: an application requests information from an external service, receives structured JSON data, validates and processes the response, extracts required fields, and uses the result inside the application. These concepts form a foundation for working with real-world APIs such as weather, maps, payment, AI, social media, and other web services.

Project Status: Completed Successfully